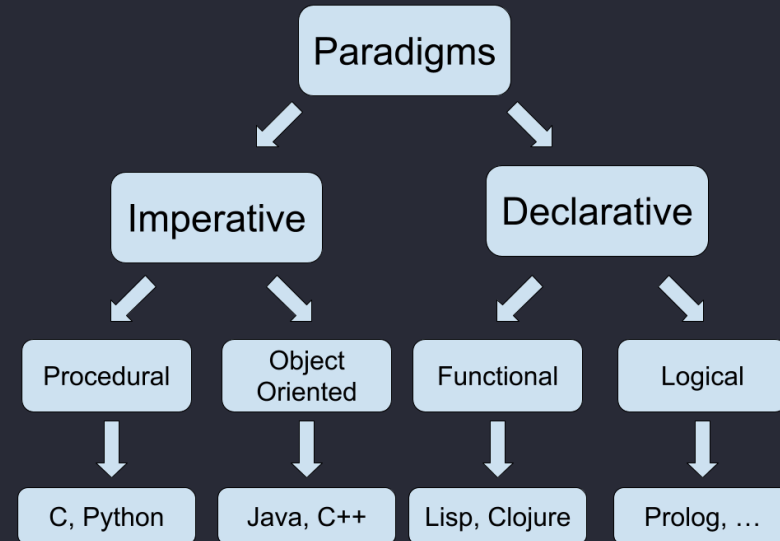


Intro to Lisp

CS-3180

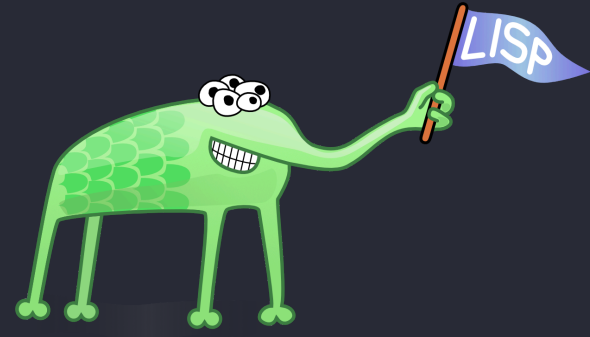
Paradigms

- We have discussed a few languages now:
- Python, Java
- JS, Rust, etc
- Most are imperative
- True for most languages you will use



Lisp!

- Functional Programming Language
- Primary paradigm*
- Much harder to learn
- Fundamentally different mental model
- Slow build up



New Language

- How to learn a new language?
- Python doesn't count
- Let's determine what we are working with
- Before we start programming in it

Complications

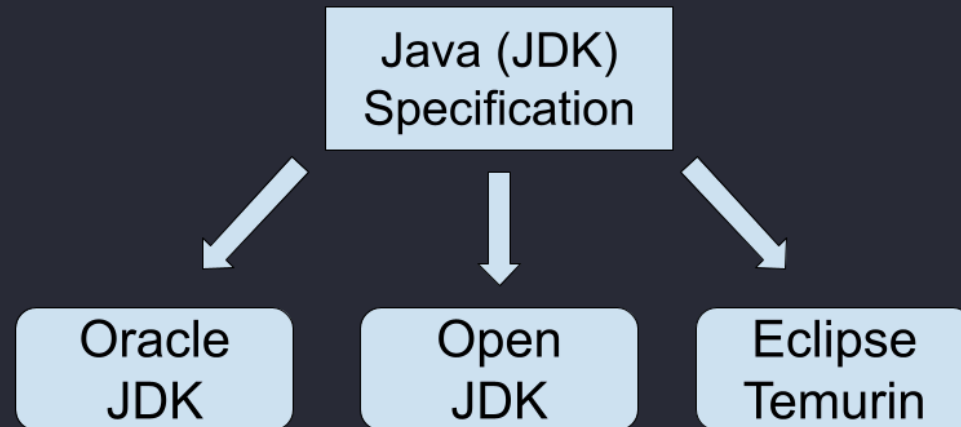
- Lisp is really a family of languages
- There are many flavors of Lisp
- Created in 1958
- Gone through many modernizations
- How is this even defined?
- How are languages defined in general

Language Specifications

- Most languages are actually not the executor
- A language is just its specification
- Usually a 100 pg. PDF file
- Maintained by ISOs
- Some have a "reference implementation"
- "This impl. works as it should, build to that spec"

Language Specifications

- Specifications are often made after the initial implementation
- Many implementations of that standard
- Each with their own quirks

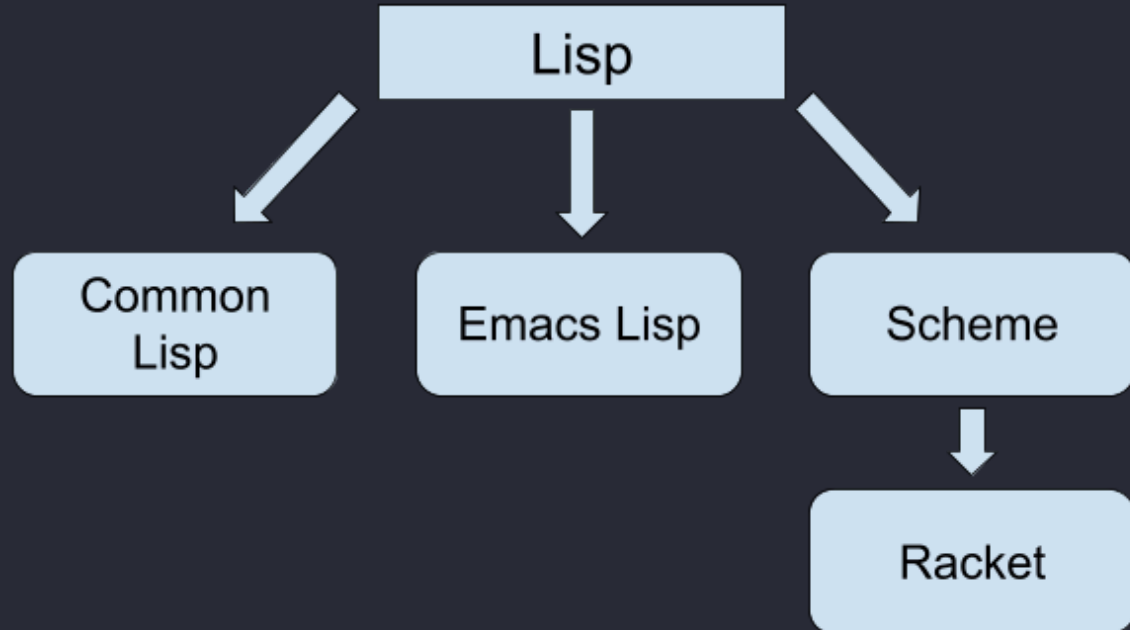


"Development Kit"

- Overly complicated for Java
- JDK: series of binaries
- Some implement just pieces
- Most languages: implementation of the executor
- When implementations build off each other, we call that a "dialect" of the other language

Lisp Languages

- Lisp Family
- Not Implementations
- Racket is a dialect of Scheme, etc
- We will be using CommonLisp



Steel Bank CL

- We will be using Steel Bank Common Lisp
- Install from package managers (Windows users: check SBCL download)
- FOSS CL implementation
- The CL specification does not state how CL should be executed
- Can either be compiled or interpreted
- SBCL supports both*

SBCL

- `sbc l` will give us a REPL
- REPL driven development
- Most projects are not actually REPL based
- Take a look at `lisp.sh` from your next lab

lisp.sh

```
1 #!/bin/bash
2
3 # LISP execution script based on AUR package:
4 # https://github.com/xyproto/lisp
5
6
7 # Goal with this is to sort of eliminate the REPL feel for st
8 # irl lisp programs are almost never ran as a REPL
9
10 clear
11
12 LISP_INTERPRETER=sbcl
13
14 if ! command -v $LISP_INTERPRETER >/dev/null 2>&1; then
15     echo "sbcl could not be found"
16     echo "Please ensure it is installed before running"
```

lisp.sh

```
7 # Goal with this is to sort of eliminate the REPL feel for st
8 # irl lisp programs are almost never ran as a REPL
9
10 clear
11
12 LISP_INTERPRETER=sbcl
13
14 if ! command -v $LISP_INTERPRETER >/dev/null 2>&1; then
15     echo "sbcl could not be found"
16     echo "Please ensure it is installed before running"
17     exit 1
18 fi
19
20
21 if [ -z $1 ]; then
22     echo "Please provide a filename as the first argument"
23     exit 1
```

lisp.sh

```
14 if [ -z $(command -v sbcl) ]; then
15     echo "sbcl could not be found"
16     echo "Please ensure it is installed before running"
17     exit 1
18 fi
19
20
21 if [ -z $1 ]; then
22     echo "Please provide a filename as the first argument"
23     exit 1
24 fi
25
26 if [ ! -f $1 ]; then
27     echo "Could not find the file named \"$1\""
28     exit 1
29 fi
30
31 # force sh to run in interpreter mode
```

lisp.sh

```
20
21 if [ -z $1 ]; then
22     echo "Please provide a filename as the first argument"
23     exit 1
24 fi
25
26 if [ ! -f $1 ]; then
27     echo "Could not find the file named \"$1\""
28     exit 1
29 fi
30
31 # force sbcl to run in interpreter mode
32 # this is 100% slower, but more in line with traditional lisp
33
34 $LISP_INTERPRETER \
35 --noinform \
36 --no-print \
```

lisp.sh

```
30
31 # force sbcl to run in interpreter mode
32 # this is 100% slower, but more in line with traditional lisp
33
34 $LISP_INTERPRETER \
35 --noinform \
36     --noprint \
37     --no-userinit \
38     --no-sysinit \
39     --disable-debugger \
40     --quit \
41     --eval '(setf sb-ext:*evaluator-mode* :interpret)' \
42     --load "$1" \
43     --end-toplevel-options
44 # this is not in the help menu, but is documented at http
45
```

lisp.sh

```
1 #!/bin/bash
2
3 # LISP execution script based on AUR package:
4 # https://github.com/xyproto/lisp
5
6
7 # Goal with this is to sort of eliminate the REPL feel for st
8 # irl lisp programs are almost never ran as a REPL
9
10 clear
11
12 LISP_INTERPRETER=sbcl
13
14 if ! command -v $LISP_INTERPRETER >/dev/null 2>&1; then
15     echo "sbcl could not be found"
16     echo "Please ensure it is installed before running"
```

lisp.sh

- Mostly to simplify the mental model
- Lisp can be interpreted, but needs "macro expansion time"
- `./lisp.sh main.lisp`
- Instead of copy-pasting into a REPL

Lists

- LISP = LISt Processor
- EVERYTHING in Lisp is a list
- The data AND the code
- SBCL evaluates Lists
- Dive into a Hello World
- Including the set-up

Running Lisp

- I prefer `.lisp` extension
- Also see `.lsp` or `.cl`
- Preferred setup:

```
□ MyProject
├─ lisp.sh
├─ src
│   └─ Main.lisp
```

- `./lisp.sh src/Main.lisp`

Hello world

- Parentheses are EXTREMELY important
- Cannot use them wherever
- `format`
- `t` for standard out
- `~%` for new line

```
; semi-colon for comments :(  
; common lisp hello world  
(format t "Hello, world!~%")
```

Case Insensitivity

- Casing doesn't matter in Lisp
- Technically converts everything* to uppercase
- I recommend lowercase
- Make it easy on your syntax highlighter

```
(format t "Hello, world!~%")  
(FORMAT T "Hello, world!~%")  
(fOrMat T "Hello, world!~%")
```

What's going on here?

- `t` is effectively a primitive type
- `t` and `nil` are the basic types
- Most things are a list*
- (Lisp actually has a much more interesting sub-typing system)
- Lisp is strong and dynamically typed

Prefix Notation

- We commonly use *infix* notation
- Operator is between values:
- $((1 * 3) + (2 + 2))$
- Lisp uses *prefix* notation:
- Operator comes before the values
- $(+ (* 1 3) (+ 2 2))$

Code as Lists

- Lisp code is a series of nested parentheses
- `~S` print in readable format
- Really pay attention to these parentheses
- Cardinal sin of Lisp formatting:
- "A line shall never consist of only parentheses"

```
(format t "~S~%"  
        (* (+ 1 3) 4))
```

Why???

- Why on earth would you want this?
- Less ambiguity:
- $8 / 2 (2 + 2)$
- All operators are functions
- Much easier to parse

Notes

- Lisp doesn't really have reserved words
- "Keywords" don't exist in the traditional sense
- *Technically there are KEYWORDS, but those are loaded at runtime and not built into the spec.
- Basically everything is a library function/macro
- +, /, <, >, =

Summary

- Lisp is a family of functional languages
- CommonLisp is one specification
- SBCL is the implementation
- Lisp uses prefix notation
- Basically everything is a function/macro

Questions